FOUNDATION OF JAVA LAB SHEET – 7
Practice Questions Multithreading

1. (Easy) Write a Java program that creates a thread by extending the Thread class and prints your name 5 times with a 200 ms delay between each print.

```
class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Sumit");
            try {
                Thread.sleep(200);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start();
    }
}
```

OUTPUT:
Sumit
Sumit
Sumit
Sumit
Sumit

2. (Easy) Rewrite Question 1 using the Runnable interface instead of extending Thread.

```
class MyTask implements Runnable {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Sumit");
```

```
            try {
                Thread.sleep(200);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Thread t = new Thread(new MyTask());
        t.start();
    }
}
```

OUTPUT:
Sumit
Sumit
Sumit
Sumit
Sumit

3. (Easy) Write a program that prints the name and priority of the currently executing (main) thread, then changes its name and prints it again.

```
public class Main {
    public static void main(String[] args) {
        Thread t = Thread.currentThread();

        System.out.println("Name: " + t.getName());
        System.out.println("Priority: " + t.getPriority());

        t.setName("MyMainThread");

        System.out.println("New Name: " + t.getName());
    }
}
```

OUTPUT:
Name: main
Priority: 5
New Name: MyMainThread

4. (Easy) Create two threads that each print their own name and a count from 1 to 5. Run the program three times and note whether the output order changes each time.

```
class MyThread extends Thread {
    MyThread(String name) {
        super(name);
    }

    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(getName() + ": " + i);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread("Thread-1");
        MyThread t2 = new MyThread("Thread-2");

        t1.start();
        t2.start();
    }
}
```

OUTPUT:

```
Thread-1: 1
Thread-1: 2
Thread-2: 1
Thread-2: 2
Thread-1: 3
Thread-2: 3
Thread-1: 4
Thread-2: 4
Thread-1: 5
Thread-2: 5
```

5. (Moderate) Write a program with three threads, each named "Reader", "Writer", and "Logger". Each should sleep for a random time (0-500 ms) before printing its name and a message.

```
class MyThread extends Thread {
    MyThread(String name) {
        super(name);
    }

    public void run() {
        try {
            int delay = (int)(Math.random() * 501);
            Thread.sleep(delay);
            System.out.println(getName() + " - Task completed");
        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        new MyThread("Reader").start();
        new MyThread("Writer").start();
        new MyThread("Logger").start();
    }
}
```

OUTPUT:
Writer - Task completed
Reader - Task completed
Logger - Task completed

6. (Easy) Write a program that demonstrates the use of isAlive() to check a thread's status before it starts, while it runs, and after it finishes.

```
class MyThread extends Thread {
    public void run() {
        try {
            Thread.sleep(500);
        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }
}

public class Main {
```

```
    public static void main(String[] args) throws InterruptedException {
        MyThread t = new MyThread();

        System.out.println("Before start: " + t.isAlive());

        t.start();
        System.out.println("While running: " + t.isAlive());

        t.join();

        System.out.println("After finish: " + t.isAlive());
    }
}
```

OUTPUT:
Before start: false
While running: true
After finish: false

7. (Moderate) Create a thread that performs a long-running task (a loop summing numbers from 1 to 10,000). Use join() in the main method to ensure the main thread prints the "Task finished" message only after the calculation completes.

```
class SumThread extends Thread {
    public void run() {
        long sum = 0;

        for (int i = 1; i <= 10000; i++) {
            sum += i;
        }

        System.out.println("Sum: " + sum);
    }
}

public class Main {
    public static void main(String[] args) throws InterruptedException {
        SumThread t = new SumThread();

        t.start();
        t.join();
```

```
        System.out.println("Task finished");
    }
}
```

OUTPUT:
Sum: 50005000
Task finished

8. (Easy) Write a program that creates a thread, sets its priority to Thread.MAX_PRIORITY, and another thread with Thread.MIN_PRIORITY. Print each thread's priority before starting them.

```
class MyThread extends Thread {
    MyThread(String name) {
        super(name);
    }

    public void run() {
        System.out.println(getName() + " is running");
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread high = new MyThread("High Priority");
        MyThread low = new MyThread("Low Priority");

        high.setPriority(Thread.MAX_PRIORITY);
        low.setPriority(Thread.MIN_PRIORITY);

        System.out.println(high.getName() + ": " + high.getPriority());
        System.out.println(low.getName() + ": " + low.getPriority());

        high.start();
        low.start();
    }
}
```

OUTPUT:
High Priority: 10
Low Priority: 1
High Priority is running
Low Priority is running

9. (Moderate) Create five threads, assign them priorities 1 through 10 in steps of roughly 2 (or any values you choose), start all of them, and observe (over multiple runs) whether higher priority threads tend to finish their print statements earlier.

```
class MyThread extends Thread {
    MyThread(String name, int priority) {
        super(name);
        setPriority(priority);
    }

    public void run() {
        for (int i = 1; i <= 3; i++)
            System.out.println(getName() + " (Priority " + getPriority() + "): " + i);
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread("Thread-1", 1);
        MyThread t2 = new MyThread("Thread-2", 3);
        MyThread t3 = new MyThread("Thread-3", 5);
        MyThread t4 = new MyThread("Thread-4", 7);
        MyThread t5 = new MyThread("Thread-5", 10);

        t1.start();
        t2.start();
        t3.start();
        t4.start();
        t5.start();
    }
}
```

OUTPUT:

```
Thread-5 (Priority 10): 1
Thread-4 (Priority 7): 1
Thread-3 (Priority 5): 1
Thread-5 (Priority 10): 2
Thread-2 (Priority 3): 1
Thread-1 (Priority 1): 1
...
```

10. (Moderate) A shared BankAccount class has a balance field and a withdraw(int amount) method (not synchronized). Create two threads that each try to withdraw money from the same account 1000 times. Run it and show that the final balance is sometimes incorrect due to a race condition.

```
class BankAccount {
    int balance = 2000;

    void withdraw(int amount) {
        if (balance >= amount) {
            int temp = balance;
            temp = temp - amount;
            balance = temp;
        }
    }
}

class MyThread extends Thread {
    BankAccount account;

    MyThread(BankAccount account) {
        this.account = account;
    }

    public void run() {
        for (int i = 1; i <= 1000; i++)
            account.withdraw(1);
    }
}

public class Main {
    public static void main(String[] args) throws InterruptedException {
        BankAccount account = new BankAccount();

        Thread t1 = new MyThread(account);
        Thread t2 = new MyThread(account);

        t1.start();
        t2.start();

        t1.join();
        t2.join();
```

```
        System.out.println("Final Balance: " + account.balance);
    }
}
```

OUTPUT:
Final Balance: 0

11. (Moderate) Fix Question 10 by making withdraw() a synchronized method, and confirm the final balance is now always correct.

```
class BankAccount {
    int balance = 2000;

    synchronized void withdraw(int amount) {
        if (balance >= amount) {
            balance = balance - amount;
        }
    }
}

class MyThread extends Thread {
    BankAccount account;

    MyThread(BankAccount account) {
        this.account = account;
    }

    public void run() {
        for (int i = 1; i <= 1000; i++)
            account.withdraw(1);
    }
}

public class Main {
    public static void main(String[] args) throws InterruptedException {
        BankAccount account = new BankAccount();

        Thread t1 = new MyThread(account);
        Thread t2 = new MyThread(account);

        t1.start();
```

```
        t2.start();

        t1.join();
        t2.join();

        System.out.println("Final Balance: " + account.balance);
    }
}
```

OUTPUT:
Final Balance: 0

12. (Moderate) Rewrite Question 11 so that instead of synchronizing the whole method, you use a synchronized block that locks only a dedicated lock object, and only around the statements that read/update the balance.

```
class BankAccount {
    int balance = 2000;
    private final Object lock = new Object();

    void withdraw(int amount) {
        synchronized (lock) {
            if (balance >= amount) {
                balance = balance - amount;
            }
        }
    }
}

class MyThread extends Thread {
    BankAccount account;

    MyThread(BankAccount account) {
        this.account = account;
    }

    public void run() {
        for (int i = 1; i <= 1000; i++)
            account.withdraw(1);
    }
```

```
}

public class Main {
    public static void main(String[] args) throws InterruptedException {
        BankAccount account = new BankAccount();

        Thread t1 = new MyThread(account);
        Thread t2 = new MyThread(account);

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println("Final Balance: " + account.balance);
    }
}
```

OUTPUT:
Final Balance: 0

13. (Moderate) Implement a simple producer-consumer scenario using wait() and notify() where a producer thread adds numbers (1 to 10) one at a time to a shared single-slot buffer, and a consumer thread reads and prints each number as it becomes available.

```
class Buffer {
    int value;
    boolean available = false;

    synchronized void produce(int n) throws InterruptedException {
        while (available)
            wait();

        value = n;
        available = true;
        System.out.println("Produced: " + n);
        notify();
    }

    synchronized void consume() throws InterruptedException {
```

```
        while (!available)
            wait();

        System.out.println("Consumed: " + value);
        available = false;
        notify();
    }
}

public class Main {
    public static void main(String[] args) {
        Buffer b = new Buffer();

        Thread producer = new Thread(() -> {
            try {
                for (int i = 1; i <= 10; i++)
                    b.produce(i);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        });

        Thread consumer = new Thread(() -> {
            try {
                for (int i = 1; i <= 10; i++)
                    b.consume();
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        });

        producer.start();
        consumer.start();
    }
}

OUTPUT:
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
...
```

Produced: 10
Consumed: 10

14. (Moderate) Write a program that creates a deadlock situation using two shared objects, Lock1 and Lock2, where two threads try to acquire them in opposite order. Run it and observe that the program never terminates (you will need to stop it manually).

```
class Main {
    static Object Lock1 = new Object();
    static Object Lock2 = new Object();

    public static void main(String[] args) {
        Thread t1 = new Thread(() -> {
            synchronized (Lock1) {
                System.out.println("Thread 1: Lock1");
                try { Thread.sleep(100); } catch (Exception e) {}

                synchronized (Lock2) {
                    System.out.println("Thread 1: Lock2");
                }
            }
        });

        Thread t2 = new Thread(() -> {
            synchronized (Lock2) {
                System.out.println("Thread 2: Lock2");
                try { Thread.sleep(100); } catch (Exception e) {}

                synchronized (Lock1) {
                    System.out.println("Thread 2: Lock1");
                }
            }
        });

        t1.start();
        t2.start();
    }
}
```

OUTPUT:
Thread 1: Lock1
Thread 2: Lock2

15. (Moderate) Fix your deadlock program from Question 14 by ensuring both threads acquire Lock1 and Lock2 in the same order, and confirm the program now completes normally.

```
class Main {
    static Object Lock1 = new Object();
    static Object Lock2 = new Object();

    public static void main(String[] args) {
        Thread t1 = new Thread(() -> {
            synchronized (Lock1) {
                System.out.println("Thread 1: Lock1");

                synchronized (Lock2) {
                    System.out.println("Thread 1: Lock2");
                }
            }
        });

        Thread t2 = new Thread(() -> {
            synchronized (Lock1) {
                System.out.println("Thread 2: Lock1");

                synchronized (Lock2) {
                    System.out.println("Thread 2: Lock2");
                }
            }
        });

        t1.start();
        t2.start();
    }
}
```

OUTPUT:

```
Thread 1: Lock1
Thread 1: Lock2
Thread 2: Lock1
Thread 2: Lock2
```